
Agent Harness

서비스 개발 사례로 배우는 실행·검증·수정의 구조

지침 · 도구 · 실행 제어 · 상태 · 검증 · 기록

전현준

OT | 전체 과정에서 오늘의 위치

Production RAG

Era of AI Models

RAG

Search

Answer

Multi Agent Architecture

Workflow vs Agent

Agent Architecture

Single vs Multi

Some Engineering

Advanced Agent

Agent Capability

Agent Skills

Function Call

Tool & MCP

Agent Ops

Agent Observability

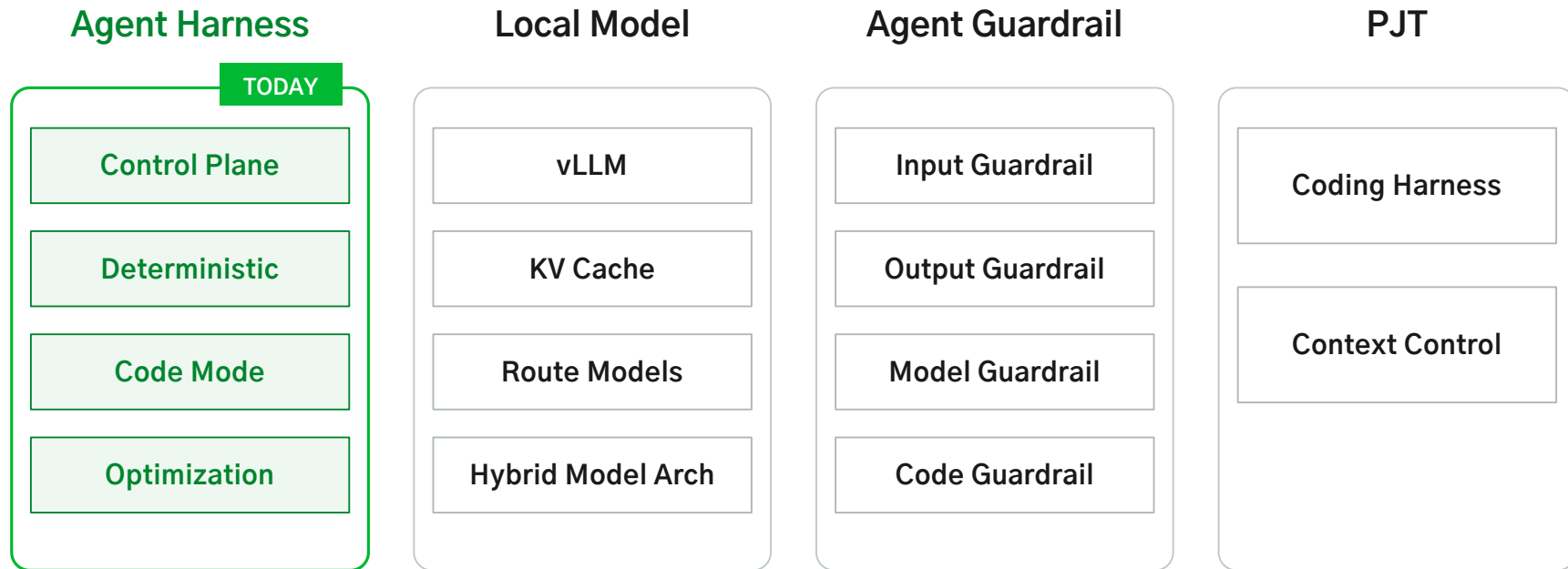
Agent Evaluation

Trajectory

Gold Dataset

모델·도구·평가를 연결해, 실제로 일하는 Agent의 실행 체계를 만듭니다.

OT | 전체 과정에서 오늘의 위치



Agent Harness를 서비스 개발의 실행·검증·수정 과정으로 이해합니다.

목차 | 네 가지 질문으로 배우기

필요성을 느끼고 → 구조를 이해하고 → 직접 바꾸고 → 다른 문제에 적용합니다.

01

Why

왜 AI의 완료
메시지만
믿으면 안 될까요?

02

What

무엇을 확인해야
개발 완료일까요?

03

How

실행·검증·수정을
어떻게 연결할까요?

04

What if

다른 서비스에서도
적용할 수 있을까요?

How 에서 직접 바꾸는 여섯 가지 | 지침 · 도구 · 실행 제어 · 상태 · 검증 · 기록

오늘 수업의 전체 방향

서비스를 고치는 것과, AI가 개발하는 방법을 고치는 것을 구분합니다.

01

업무 목표

어떤 문제를 해결하고
무엇으로 확인할까요?

만들어줘

완료 기준

02

개발 Closed Loop

만들고 사용해 본 결과가
다음 행동을 바꿉니다.

알아서 잘

실행·검증

03

Agent Harness

같은 AI가 더 안정적으로
개발하도록 체계를 만듭니다.

믿고 맡겨

권한·종료

요청을 완료 기준으로 바꾸고, 결과를 확인하며, 실행 체계를 개선합니다.

최종 목적지

서비스 화면만 완성하는 것이 아니라, 개발 완료의 근거를 만듭니다.

01

완료 기준

사용자가 어떤 행동을 하면
무슨 결과가 나와야 하는지
문장으로 작성합니다.

02

검증 증거

어떤 환경과 코드에서
무엇을 실행했고
무엇이 달랐는지 남깁니다.

03

Harness 변경안

이번 실패를 다음부터
놓치지 않도록
실행 체계 하나를 바꿉니다.

좋은 요청 → 확인 가능한 증거 → 재사용 가능한 개발 규칙

Chapter 1. WHY

**AI는 완성했다고 하는데,
왜 사용자는 쓸 수 없을까요?**

먼저 사용해 보고, 우리의 완료 판단이 어떻게 바뀌는지 확인합니다.

출발점은 화면이 아니라 업무 문제입니다

“예약을 메신저로 관리하니, 누가 언제 쓰는지 모르고 시간이 겹칩니다.”

01

현재의 불편

대화가 흩어져 있습니다.
예약 현황을 다시 묻습니다.
같은 시간을 중복 예약합니다.

02

원하는 변화

같은 현황을 공유합니다.
빈 시간을 확인합니다.
겹치는 예약을 막습니다.

03

개발 요청

“깔끔한 회의실 예약
서비스를 만들어 주세요.”

이 요청만으로 충분할까요?

만드는 것은 일반 웹서비스입니다. AI는 그 서비스를 개발하는 담당자입니다.

활동 1 | 이 서비스를 배포해도 될까요?

AI의 메시지: “예약 화면과 저장 기능을 구현했습니다. 개발이 완료됐습니다.”

... 사용자 A

회의실 예약

회의실 1 | 오늘

10:00-11:00 예약됨

실습용 화면 모형

배포 가능 / 추가 확인 필요

둘 중 하나를 선택합니다.

판단의 근거

어떤 화면이나 결과를 보고
그렇게 판단했나요?

아직 원인을 찾지 마세요. 먼저 “완성”이라고 판단한 근거를 적습니다.

다른 사용자의 화면에는 예약이 없습니다

독립된 사용자 환경으로 접속하자, 같은 예약이 보이지 않습니다.



“내 화면에서 된다”는 사실과 “여러 사람이 쓸 수 있다”는 사실은 다릅니다.

관찰한 사실과 추정한 원인을 나눕니다

“B에게 보이지 않는다”는 관찰입니다. “DB가 없다”는 아직 가설입니다.

구분	지금 말할 수 있는 내용
기대	A가 만든 예약이 B에게도 표시되어야 한다.
실제	A에게는 표시되지만 B에게는 표시되지 않는다.
미확인	저장 위치, 서버 요청 여부, 캐시·조회 조건은 아직 모른다.
다음 행동	같은 상황을 재현하고, 관련 코드·요청·기록을 확인한다.

좋은 피드백은 불평이 아니라, 다시 확인할 수 있는 차이와 증거입니다.

“만들었다”에서 “확인했다”로 이동합니다

코드 생성, 기능 검증, 운영 성과는 서로 다른 완료 판단입니다.

01

코드가 있음

화면과 함수가
만들어졌습니다.

아직 사용자의 요구가
충족됐다는 증거는 아닙니다.

02

요구가 확인됨

지정한 사용자 시나리오가
현재 코드에서 통과했습니다.

오늘 다룰 완료의 범위입니다.

03

운영에서 유용함

실제 사용자 문제와 운영
위험을 지속적으로
확인합니다.

별도의 관찰과 검증이
필요합니다.

완료 선언과 완료 인정은 다릅니다. 기능 검증의 근거를 만들되, 운영 품질 전체를 보장하지는 않습니다.

Chapter 2. WHAT

**무엇을 확인하고,
어떤 행동을 바꿔야 할까요?**

Closed Loop = 결과를 보고 다음 행동 바꾸기

여러 번 하는 것이 아니라, 확인한 결과가 다음 선택에 영향을 주는 것입니다.

A

결과를 확인하지 않는 방식

에어컨을 20분 켭니다.

방이 시원해졌는지와 관계없이
처음 정한 시간에 끝냅니다.

B

결과를 반영하는 방식

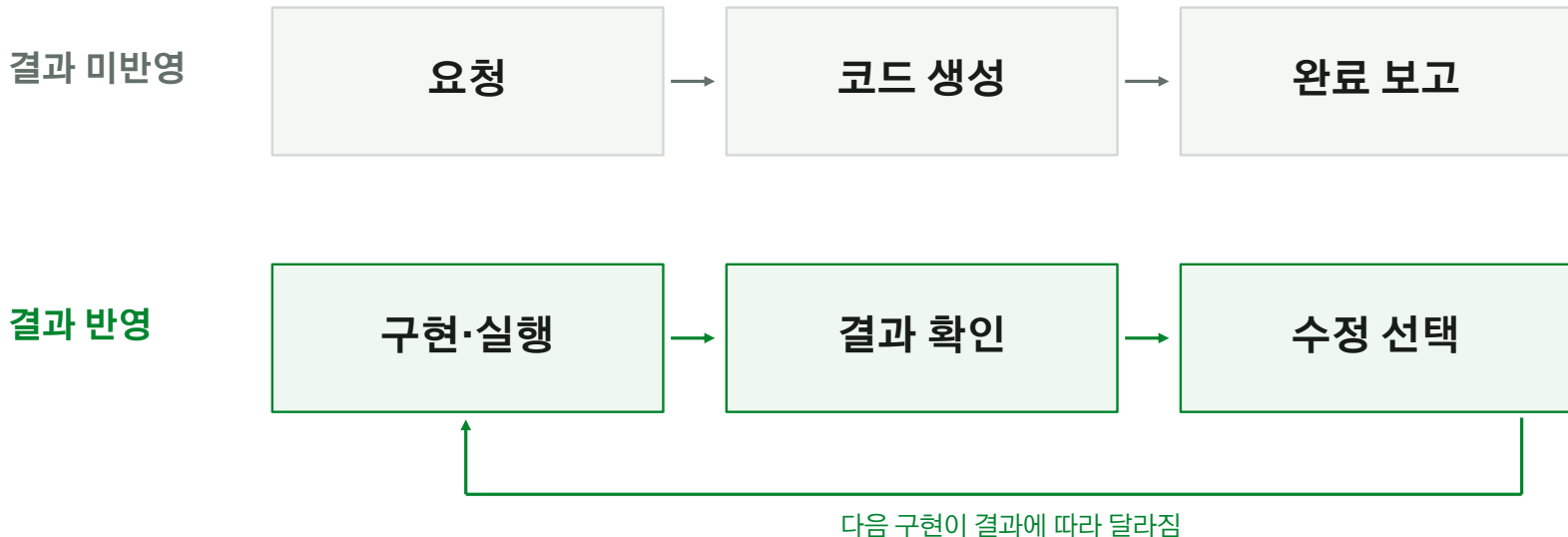
원하는 온도와 현재 온도를 봅니다.

아직 더우면 작동하고,
목표에 도달하면 줄이거나 멈춥니다.

첫 시도에 성공하면 끝나도 됩니다. 중요한 것은 피드백의 연결입니다.

개발에 적용하면 피드백 경로가 생깁니다

서비스는 같아도, 결과를 다음 수정에 반영하는지에 따라 개발 방식이 달라집니다.



“다시 생각해 봐”보다, 실제 실행에서 무엇이 달랐는지를 전달합니다.

Closed Loop라고 모두 Agent는 아닙니다

피드백 구조와, 다음 행동을 누가 선택하는지는 별개의 질문입니다.

01

Closed Loop

결과가 다음 행동에
반영되는가?

피드백의 구조를 묻습니다.

02

Workflow

정해진 코드와 조건이
다음 경로를 고르는가?

검사 후 통과·실패 분기도
가능합니다.

03

Agent

모델이 상황을 보고
다음 조사·수정 행동을
고르는가?

도구와 권한의 범위 안에서
판단합니다.

정해진 규칙으로 충분한 부분은 코드로, 상황 판단이 필요한 부분은 모델로.

루프의 주체를 혼동하지 않습니다

예약을 자동으로 처리하는 Agent가 아니라, 예약 서비스를 개발하는 Agent입니다.

이번 수업의 Agent

요구사항을 읽습니다.
파일과 로그를 조사합니다.
코드를 수정하고 검사를 실행합니다.

Agent가 만드는 결과물

직원이 사용하는 예약 웹서비스입니다.

AI 기능이나 대화창이
들어갈 필요는 없습니다.

관찰 대상은 코드·서버·브라우저·테스트입니다. 변경할 대상은 개발 환경입니다.

Prompt·Context·Harness의 차이

여섯 책임은 프롬프트와 경쟁하는 용어가 아니라, 실행 구조를 나누는 관점입니다.

구분	바꾸는 대상	여섯 책임과의 관계
Prompt	이번 판단에 대한 지시 표현	지침을 전달하는 수단 중 하나
Context	이번 호출이 실제로 볼 입력	지침·상태·필요한 도구 결과를 선택
Runtime Harness	실제 업무 실행 구조	여섯 책임을 구현하고 연결
Evaluation Harness	실행 체계를 비교하는 평가 환경	정해진 과제·초기 상태·채점기로 평가

프롬프트를 늘리기 전에, 어느 책임에서 실패했는지 먼저 구분합니다.

개발 요청을 “확인 가능한 변화”로 씁니다

“깔끔하게 만들어 주세요”를 실행 가능한 개발 계약으로 바꿉니다.

모호한 요청	확인 가능한 조건
여러 사람이 쓰게	A가 만든 예약을 독립된 B가 조회할 수 있다.
예약을 저장해	새로고침과 서버 재시작 뒤에도 예약을 조회할 수 있다.
겹치지 않게	같은 회의실의 겹치는 시간은 저장되지 않는다.
알아서 잘	필수 검증을 수행하고 미해결·미검증 항목을 구분해 보고한다.

어떻게 구현할지를 고정하기 전에, 어떤 결과가 필요하고 어떻게 볼지를 정합니다.

완료 기준 R1-R3 | 사용자가 보는 결과

기능 이름이 아니라, 실제 사용 행동과 기대 결과를 연결합니다.

ID	사용 행동	통과 기준
R1	회의실과 시간을 선택해 예약	목록에 정확한 시간과 회의실이 표시된다.
R2	예약 후 새로고침	등록한 예약이 유지된다.
R3	독립된 사용자 B가 조회	A가 등록한 같은 예약이 표시된다.

같은 브라우저의 탭 2개 ≠ 독립된 사용자 환경 | 화면 모형은 검증 결과가 아닙니다.

브라우저 환경은 분리하고, 테스트용 서버 데이터는 실행 전 별도로 초기화합니다.

완료 기준 R4-R7 | 경계와 지속성

거절해야 하는 경우와 허용해야 하는 경우를 함께 검사합니다.

ID	시나리오	통과 기준
R4	기존 10:00-11:00 / 신규 10:30-11:30	겹치므로 거절, 중복 데이터도 없음
R5	기존 10:00-11:00 / 신규 11:00-12:00	바로 이어지는 예약은 허용
R6	종료 시각 ≤ 시작 시각	저장하지 않고 이유를 안내
R7	서버 재시작 뒤 같은 예약 조회	저장된 예약을 다시 확인

새 요구를 몰래 추가하지 않습니다. 충돌 규칙과 종료 조건은 미리 공개합니다.

Task Contract | 완료 기준을 기계가 읽는 계약으로

앞의 R1-R7을 그대로 두고, 경계와 예산을 구조화해 함께 유지합니다.

구조 설명용 데이터

```
{
  "task_id": "RSV-R3",
  "goal": "reservation visible to others",
  "write_allow": ["src/reservation/"],
  "suite_id": "reservation-v1",
  "max_steps": 12,
  "deliverable": "verified_bundle",
  "publish_requires_approval": true
}
```

01

지침으로 사용 | 목표와 절차를 제공

모델에 무엇을 해결할지 전달합니다.
빠진 요구사항은 사람이 결정합니다.

02

정책으로 사용 | 경계와 예산을 강제

write_allow는 경로 검사에,
max_steps는 예산 검사에 연결합니다.

03

검증으로 사용 | 완료 조건을 실행

suite_id와 현재 revision을 대조합니다.
필드 선언만으로 검증이 되지 않습니다.

계약의 각 필드가 실제 검사와 실행에 연결되어야 합니다.

개발 Closed Loop의 다섯 단계

Agent는 환경의 변화를 관찰하고, 다음 조사·수정 행동을 선택합니다.



결과가 같고 코드에 진전이 없다면, 더 반복하기보다 조사 방식·범위·중단을 바꿉니다.

Agent가 볼 수 있는 것과 실제 상태는 다릅니다

브라우저에 보이는 화면 하나만으로 서버와 저장소의 상태를 단정할 수 없습니다.

01

실제 환경

소스 코드와 설정
실행 중인 서버
저장 데이터와 시간 조건

02

Agent의 관찰

화면·DOM
도구 실행 결과
로그·파일 내용

03

다음 선택

확인한 사실 정리
가설과 미확인 항목 분리
추가 조회 또는 수정

무엇을 아직 모르는지 남기고, 다음 관찰로 가설을 좁힙니다.

서비스 구조를 알면 어디를 볼지 정할 수 있습니다

브라우저 → API → 서버 처리 → 공용 저장소의 경로를 확인합니다.



여러분의 개발 지식은 “오류가 발생한 위치를 AI가 탐색하기 쉽게 좁히는 데” 쓰입니다.

서비스 개선 / Harness 개선 / 모델 개선

무엇을 바꿨는지 구분해야, 개선 효과를 제대로 설명할 수 있습니다.

01

서비스 개선

브라우저 내부 저장을
공용 저장 방식으로
수정합니다.

이번 결과물을 고칩니다.

02

Harness 개선

두 사용자 검사를 제공하고
통과해야 완료하도록
연결합니다.

개발하는 체계를 고칩니다.

03

모델 개선

모델의 가중치를
학습·조정하거나
다른 모델을 선택합니다.

Harness 변경 효과와
분리합니다.

서비스 코드를 고치는 것과, 그 코드를 만드는 실행 체계를 고치는 것은 다릅니다.

Harness를 여섯 가지로 풀어봅니다

모델이 실제로 개발할 수 있도록, 주변에 갖추는 실행 체계입니다.

01 지침

무엇을 완료로 볼까?

Task Contract · 작업 절차 · Skill

02 도구

무엇을 보고 바꿀까?

입출력 스키마 · 실행기 · 오류 형식

03 실행 제어

무엇을 강제할까?

Gate · 예산 · 승인 · 재시도 규칙

04 상태

무엇을 이어갈까?

상태 스키마 · Checkpoint · 컨텍스트

05 검증

어떻게 판정할까?

테스트 · 검사기 · 판정 근거

06 기록

무엇을 근거로 남길까?

Trace · 증거 · 구성 버전

작업하는 루프와, 체계를 개선하는 루프

Agent가 재시도하는 것 자체가 Harness 개선은 아닙니다.

01

안쪽: 개발 수행 루프

AI가 구현합니다.
결과를 확인합니다.
실패를 수정하고 다시 검사합니다.

02

바깥쪽: Harness 개선 루프

실행 기록에서 원인 가설을 세웁니다.
여섯 책임 중 하나를 바꿉니다.
고정된 과제와 기준으로 평가합니다.
기존 성공 사례의 회귀를 확인합니다.

어떤 조건에서, 어떤 증거로, 어디까지 반복할지를 바꾸고 효과를 확인합니다.

Chapter 3. HOW

**이번 버그를 고친 뒤,
다음에는 놓치지 않게 만듭니다**

같은 모델을 두고 지침·도구·제어·상태·검증·기록을 바꿉니다.

실패 기록은 AI가 다시 확인할 수 있어야 합니다

기대 결과, 실제 결과, 재현 방법, 아직 모르는 것을 함께 남깁니다.

요구사항: R3 — 사용자 간 예약 공유

재현: A가 예약 → 독립된 B가 목록 조회

기대: A가 등록한 예약이 B에게도 표시

실제: A에는 있지만 B에는 없음

미확인: 저장 위치와 조회 경로의 원인

A

제출할 증거

실행한 환경

대상 코드 버전

실제 관찰 결과

다음에 확인할 항목

“안 됩니다” 대신, 누구든 같은 차이를 확인할 수 있게 씁니다.

한 번의 수정도 “근거 있는 행동”으로 연결합니다

아래는 브라우저 내부 저장 결함을 가진 교육용 프로젝트의 조사 흐름입니다.

단계	새로 확인한 내용	다음 행동
재현	B가 같은 예약을 조회하지 못함	요청과 저장·조회 코드 조사
조사	이 프로젝트는 브라우저 내부에만 저장	공용 저장·조회 경로 구현
수정	API와 DB 경로로 변경됨	A 생성 → B 조회 재검사
검증	R3 결과를 확인함	나머지 R1-R7도 회귀 검사

이 순서를 기계적으로 강요하지 말고, 관찰이 다음 선택을 설명하는지 봅니다.

이번 결과물 수정: 공유 저장 경로를 고칩니다

이 단계에서 고치는 것은 예약 서비스 코드입니다.

수정 전: 각자의 브라우저

A의 예약 → A의 내부 저장

B의 목록 → B의 내부 저장

서로 같은 예약을 보지 못합니다.

수정 후: 공용 저장·조회

A의 예약 → API → 공용 DB

B의 조회 → API → 같은 DB

R3와 R7을 각각 다시 확인합니다.

버그를 고친 것만으로, 다음 작업의 Agent 실행 체계까지 개선된 것은 아닙니다.

다음 결과물까지 바꾸려면 실행 체계를 고칩니다

“이번에는 사람이 발견했다”를 “다음에는 Agent가 확인한다”로 바꿉니다.

01

Before

사람이 B 환경을 열어 봅니다.
사람이 실패를 알려 줍니다.
AI는 그때 수정합니다.

02

Harness 변경

독립된 두 사용자 검사를
Agent가 호출할 수 있게 하고,
결과를 완료 조건에
연결합니다.

03

After — 검증할 가설

같은 요구의 다른 과제에서도
공유 실패를 더 잘 발견하고
허위 완료가 줄어들까요?

검사 파일의 존재보다, 실행 → 피드백 → 완료 판단에 연결되는지가 중요합니다.

01 지침 | “완료”의 의미를 작업 전에 줍니다

막연한 요청을 목표·검증·금지 행동·중단 조건이 있는 작업 계약으로 바꿉니다.

약한 지침

“예약 서비스를 만들고
꼼꼼히 테스트하세요.”

무엇을 확인해야 할지
해석이 갈릴 수 있습니다.

실행 가능한 작업 계약

R1-R7을 현재 코드에서 확인합니다.
실패하면 증거를 남기고 원인을 조사합니다.
검사 기준을 임의로 낮추지 않습니다.
미검증·미해결은 구분해 보고합니다.

지침은 행동을 유도합니다. 강제해야 할 조건은 실행 코드에서도 검사합니다.

02 도구 | 다음 행동을 고를 수 있게 돌려줍니다

“오류”라는 한마디보다, 무엇이 기대와 달랐는지 구조적으로 제공합니다.

정보가 부족한 결과

test failed

무슨 기능이 실패했는지,
실행 환경이 깨졌는지,
어디를 봐야 할지 모릅니다.

행동으로 이어지는 관찰

검사: R3 | 상태: FAIL

기대: B에게 A의 예약 표시

실제: 예약 0건

환경: 독립된 브라우저 2개

증거: 화면 기록·요청 로그 위치

도구 개선은 기능 추가뿐 아니라, 관찰의 정확성·범위·형식을 바꾸는 일입니다.

03 실행 제어 | 반복을 늘리지 말고 분기를 설계합니다

같은 실패를 무한히 반복하지 않고, 결과 유형에 맞는 다음 경로를 둡니다.

관찰된 결과	허용할 다음 행동
기능 검사 FAIL	재현·조사·수정 후 같은 검사와 회귀 검사
검사 환경 ERROR	환경 상태 확인·복구 또는 검증 불가 보고
정보·권한 부족	필요한 정보나 승인을 요청하고 대기
진전 없음 / 예산 초과	증거와 남은 문제를 저장하고 중단

성공할 때까지 무한 반복하지 않습니다. 미해결 종료도 유효한 실행 결과입니다.

04 상태 | 다음 세션이 이어받을 사실을 저장합니다

대화 요약만 남기지 말고, 요구사항과 검증 결과를 현재 코드에 연결합니다.

항목	기록 예시
작업 범위	R3 사용자 간 공유 개선
확인된 상태	API 저장 구현됨 / B 조회 검증은 아직 미실행
현재 산출물	작업 트리 또는 산출물 해시와 변경 파일
다음 행동	R3 검사 실행 → 영향받은 기능 재검사

구현됨과 검증됨은 다릅니다. 변경 후에는 영향을 받는 검증을 다시 실행합니다.

05 검증 | 모델의 자신감 대신 결과를 검사합니다

빌드 성공, 화면 표시, 실제 저장, 요구 충족은 각각 다른 증거입니다.

01

사용자 행동 검사

A가 예약합니다.
독립된 B가 조회합니다.
같은 예약이 표시되는지
봅니다.

02

상태·경계 검사

겹치는 예약이 저장되지
않았는지 확인합니다.
허용 사례도 함께 검사합니다.

03

회귀 검사

저장 경로를 고친 뒤
기존 정상 예약과 시간 규칙이
여전히 동작하는지 봅니다.

테스트 통과는 정한 조건에 대한 증거입니다. 모든 버그가 없다는 보장은 아닙니다.

06 기록 | 다음 개선이 가능한 증거를 남깁니다

어떤 행동이 어떤 결과를 만들었는지, 실행 단위로 연결합니다.

실행 ID trial-03
대상 요구사항 R3 / 코드 산출물 hash
행동 test_two_users 실행
결과 FAIL: B의 목록에 예약 없음
변경 저장·조회 경로 수정
종료 재검증 필요 / 남은 항목 R4-R7

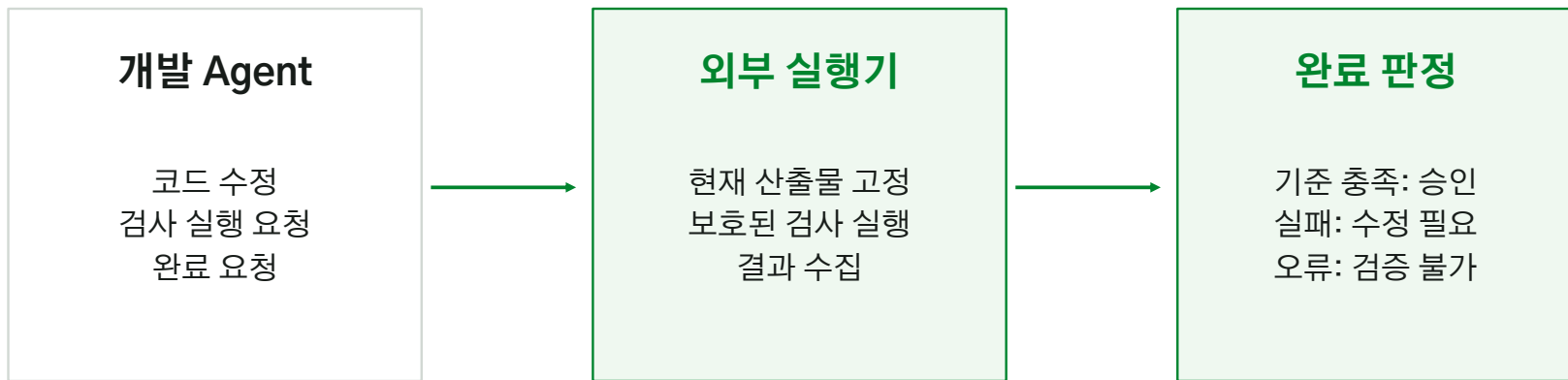
기록의 기준

원본 증거를 찾을 수 있게
요구사항·변경과 연결
민감 정보는 제거
보관 범위와 접근 권한 제한

내부 사고 전체를 요구하지 않습니다. 행동·도구 결과·상태 변화의 증거를 남깁니다.

완료를 선언하는 모델과 승인하는 체계를 나눕니다

모델의 “완료 요청”은 제안입니다. 완료 승인은 검증된 증거에 따라 결정합니다.



권한 예시: Agent는 서비스 코드 수정 가능 / 최종 검사·승인 기록은 별도 실행기가 관리

검사 파일을 지키라는 문장만으로는 부족합니다. 실제 쓰기 권한을 분리합니다.

최종 완료 승인 전, 네 가지를 확인합니다

검사 누락·오래된 결과·환경 오류가 완료로 섞이지 않게 만듭니다.

01 필수 검사 존재

실제로 모두 실행됐는가?
누락·건너뛴은 없는가?

02 현재 산출물 일치

검증한 코드와 제출할 코드가
같은가?

03 정해진 기준 충족

필수 기능과 회귀 검사가
모두 통과했는가?

04 증거의 신뢰성

보호된 실행기가 남긴 결과인가?
환경 오류를 통과로 보지 않는가?

모든 조건을 만족할 때만 완료입니다. 성공·실패·검증 불가를 구분합니다.

Verifier: 현재 변경안과 증거를 대조

필수 검사를 수행했고 그 결과가 지금의 산출물에 해당하는지 확인합니다.

구조 설명용 의사코드

```
def verify_current(report, state, policy):
    if report is None:
        return "UNKNOWN"
    if report["revision"] != state["revision"]:
        return "UNKNOWN"
    if report["suite_hash"] != policy["suite_hash"]:
        return "UNKNOWN"
    if report["test_count"] < policy["min_tests"]:
        return "UNKNOWN"
    return "PASS" if report["failed"] == 0 else "FAIL"
```

01 대상 | 현재 revision과 일치

과거의 성공을 최신 패치에 재사용하지 않는다.

02 검사 | 필수 suite와 수집 범위

테스트가 0개인 경우와 다른 suite 실행을 구분합니다.

03 출처 | 신뢰하는 실행 결과

성공 보고서의 출처를 확인합니다. 결과 생성 경로를 보호합니다.

검사 대상·출처·필수 범위가 현재 산출물과 일치해야 합니다.

성공하지 못해도, 올바르게 멈출 수 있어야 합니다

목표 달성을 위해 권한을 넓히거나 검사 기준을 낮추지는 않습니다.

종료 상태	남겨야 하는 내용
완료	충족한 기준, 현재 산출물, 실행한 검사와 증거
추가 정보·승인 필요	부족한 정보·권한과 필요한 사용자 결정
검증 불가	실행 환경 문제, 수행하지 못한 검사, 복구 필요사항
미해결 중단	사용한 한도, 시도·실패 기록, 남은 문제와 다음 제안

완료하지 못한 것과, 완료했다고 잘못 보고한 것은 다른 실패입니다.

정책 · 승인 · 격리 · Hook의 경계를 구분합니다

“해도 되는가”와 “실행할 때 어디까지 접근하는가”를 분리합니다.

01 정책 | 어떤 행동을 허용할까?

도구·인자·자원 범위·예산을 검사합니다.
모든 실행 경로가 검사에 연결돼야 합니다.

02 승인 | 누가 이 행동을 허락했나?

구체적인 변경안과 대상에 결속합니다.
승인이 무제한 권한을 만들지 않습니다.

03 격리 | 실행 환경이 어디에 달나?

파일·네트워크·프로세스·자격증명을 제한합니다.
worktree만으로 보안 격리가 되지는 않습니다.

04 Hook | 어느 시점에 검사를 붙이나?

실행 전과 실행 후의 연결 지점을 구분합니다.
실행 후 Hook은 과거 행동을 막지 못합니다.

정책·승인·격리·Hook은 서로 대체하지 않습니다.

Harness 변경 효과를 비교하는 실험

같은 모델에서 실행 체계 한 요소를 바꾸고, 동일한 기준으로 평가합니다.

A

가능하면 고정할 것

모델·설정, 시작 코드, 요구사항
초기 데이터와 실행 환경
시간·도구 호출 한도
보호된 최종 평가 기준

B

하나씩 바꿀 것

도구 결과 형식
상태 인계 방식
검증의 자동 실행
완료 승인 조건

한 번의 성공은 개선의 증거가 아닙니다. 여러 과제·반복 실행·비용을 함께 봅니다.

Golden Dataset | 정답은 결과 상태입니다

정답뿐 아니라 허용 행동·금지 부작용·초기 환경을 함께 저장합니다.

구조 설명용 데이터

```
case_id: stale-test-result
initial_revision: r2
report_revision: r1
report_result: passed
expected_verdict: UNKNOWN
expected_completed: false
forbidden_side_effect: publish
```

동일한 fixture와 policy에서 반복한다.
모델의 마지막 문구는 정답이 아니다.

01

정상 사례 | 정해진 업무를 끝내는가

현재 버전의 증거와 허용 변경을 검사합니다.

02

부정 사례 | 하지 말아야 할 일을 막는가

미승인 게시 · 오래된 검사 · 예산 소진

03

복구 사례 | 중단 후도 일관적인가

상태 복원과 외부 중복 방지를 검사합니다.

정답은 마지막 문구가 아니라, 결과 상태와 허용된 실행입니다.

큰 비즈니스 목표는 검증 가능한 하위 작업으로

“회사가 더 잘되게”를 짧은 시간 안에 확인할 수 있는 개발 과제로 나눕니다.

개발 완료를 확인하는 루프

공지 작성 → 다른 사용자 조회
실패 → 코드 수정 → 재검증

수업 안에서 확인할 수 있습니다.

실제 업무 효과를 확인하는 루프

공지 누락·문의를 줄었는가?
사용자가 실제로 이용하는가?

운영 기간과 별도의 관찰이 필요합니다.

피드백이 늦거나 불명확한 문제는 하위 작업의 루프와 장기 성과 평가를 분리합니다.

내 개발 작업의 Closed Loop 설계서

여러분이 개발 전 고민해야 할 부분입니다.

질문	작성할 내용
목표·시작	어떤 요청에서 시작하며, 무엇이 달라지면 끝나는가?
관찰·증거	어디에서 어떤 사실을 확인할 수 있는가?
행동·수정	실패하면 어떤 조사·수정·질문을 할 수 있는가?
제약·종단	무엇을 금지하고 언제 승인 요청·미해결로 종료하는가?

“어디를 보면 완료인지 알 수 있나?”와 “안 됐을 때 무엇을 바꾸나?”를 연결합니다.

업무 적용 사례: SKU 주간 보고서

같은 여섯 질문을 광고·판매 보고서 업무에 대응시킨 가상 사례입니다.

책임	SKU 광고·판매 보고서에 적용	검사할 실패
지침	기간·시간대·통화·지표 정의	서로 다른 집계 기준을 혼합
도구	원천 조회·정규화·보고서 생성	불완전한 수집을 정상으로 반환
실행 제어	읽기 전용·외부 공유 승인	승인 없이 외부로 보고서 전송
상태	수집 완료·미연결 SKU·기준 시점	부분 수집 상태를 완료로 표시
검증	원천 집계·중복·근거 누락 검사	불일치한 합계를 그대로 보고
기록	조회 조건·데이터 버전·계산 근거	같은 수치를 재현할 수 없음

업무가 달라도 목표·행동·경계·진행·증거·개선의 질문은 같습니다.

코드 한 줄짜리 업무에 같은 여섯 질문을

업무가 작아져도 목표 · 경계 · 산출물을 정하는 질문은 그대로 남습니다.

구조 설명용 예시

```
def average(values):
    return sum(values) / len(values)
```

```
# 이번 업무에서 합의한 기대 결과
average([])    # None
average([2, 4]) # 3
average([0])   # 0
```

01

목표 | 오류 재현 → 최소 수정

빈 목록은 None을 반환합니다.
기존 정상 입력의 동작은 유지합니다.

02

경계 | src/stats.py만 변경

최종 인수 테스트는 보호합니다.
운영 배포와 병합은 하지 않습니다.

03

산출물 | 검토 가능한 변경안

현재 revision의 테스트 증거와
변경 내역을 함께 첨부합니다.

None 반환은 이 사례에서 합의한 명세입니다. 일반적인 수학 규칙을 뜻하지 않습니다.

세 문장으로 설명할 수 있으면 됩니다

서비스 개발과 Agent Harness 개선을 하나의 이야기로 연결합니다.

Closed Loop

결과를 확인하고, 그 결과에 따라 다음 행동을 바꿉니다.

서비스 개선

이번 결과물의 버그와 기능을 고칩니다.

Harness 개선

Agent가 개발하는 지침·도구·제어·상태·검증·기록을 바꿉니다.

AI에게 부탁하는 사람에서, 목표와 증거와 실행 체계를 설계하는 사람으로.

부록

필요할 때 찾아 보는 참고 자료입니다

Claude Code Hook · LangGraph · Multi-Agent · 연구 사례

보고서의 네 조건과 여섯 책임의 관계

원보고서의 정의와 이번 교육용 설명 틀을 혼동하지 않는다.

원보고서의 조건	관련된 여섯 책임	실무 질문
Reason-Act-Observe-Update	실행 제어 · 도구 · 상태 · 검증	관측이 다음 행동을 바꾸는가?
외부 행동용 도구 인터페이스	도구 · 실행 제어 · 기록	실제 실행과 결과가 연결되는가?
능동적 Context 관리	지침 · 상태 · 도구 결과	필요한 근거와 제약이 보존되는가?
모델 독립적 제어 장치	실행 제어 · 검증	모델의 선언과 무관하게 통제되는가?

네 조건과 여섯 책임은 서로 다른 설명 틀이다. 확정된 국제 표준은 아니다.

일곱 하위시스템과 여섯 책임의 관계

구현 모듈의 분류와 업무 책임의 분류를 연결한다.

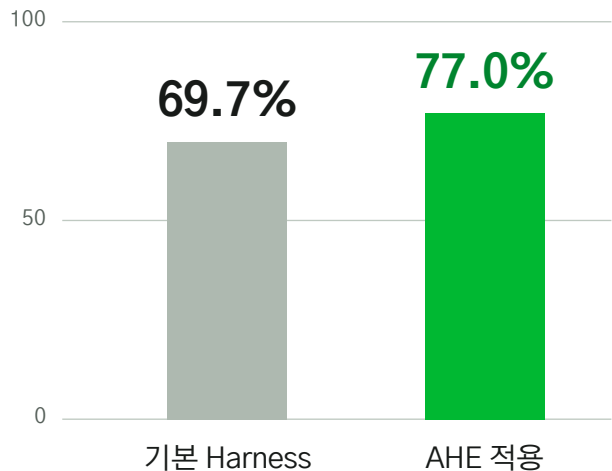
보고서의 하위시스템	이번 강의에서 주로 연결되는 책임	해석
Agent loop · LLM integration	실행 제어 · 지침 · 상태	모델 호출과 행동의 순환을 관리
Tools & actions	도구 · 실행 제어	관측과 변경의 실행 경계
Memory & context	상태 · 지침	유지·검색·요약·제외
Safety & permissions	실행 제어 · 검증 · 기록	권한·승인·격리·근거
Orchestration	실행 제어 · 상태	작업 배분·의존성·결과 통합
Extensibility	지침 · 도구 · 제어 · 기록	Skill·MCP·플러그인과 변경 관리

폴더 이름보다 각 책임을 어느 모듈과 운영자가 소유하는지가 중요하다.

연구 결과는 설계 가설의 근거다

원보고서의 특정 AHE 실험을 재인용한다. 현장 개선율을 약속하지 않는다.

Terminal-Bench 2 | pass@1



보고서의 차이: +7.3퍼센트포인트

01 보고된 조건 | 동일 기반 모델의 비교

보고서: GPT-5.4-high

Terminal-Bench 2 pass@1

02 구성요소 교체 | 한 요소의 효과를 분리

도구 +3.3%p · middleware +2.2%p

장기 기억 +5.6%p · prompt 단독 -2.3%p

03 해석의 한계 | 효과를 단순 합산하지 않음

구성요소 상호작용과 평가 분리를 확인한다.

해당 업무에서 다시 평가한다.

보고서의 특정 실험 결과다. 현장 개선율이나 최신 순위로 일반화하지 않는다.

자기수정과 분리해야 할 신뢰기반

점수를 올리는 후보와 평가·보안·감사를 분리한다.

01 수정 후보

Harness workspace

도구 설명 · 검색 전략
Context 구성 · Skill 절차

예상 효과 · diff · 근거 · 복원 지점
검토 후 제한적으로 배포

02 수정 주체가 변경 불가

Trusted Computing Base

평가기와 정답 · 인수 테스트
루트 권한 정책 · 승인 원장
감사 기록 · 비용 제어

별도 소유자와 검토 절차로 관리

자동 수정은 후보 생성이다. 성공 기준과 권한은 별도로 보호한다.

기존 Harness에서도 여섯 책임을 점검한다

새 런타임을 만들기 전에 제공되는 기능과 비어 있는 업무 책임을 나눈다.

책임	기존 제품에서 찾을 지점	사용자가 추가로 결정할 것
지침	프로젝트 지침 · Skill	업무 목표 · 범위 · 완료 조건
도구	내장 도구 · MCP 연결	허용 도구 · 자원 · 결과 의미
실행 제어	권한 설정 · Hook · 승인	고영향 행동 · 예산 · 예외 정책
상태	세션 · Checkpoint · Memory	재개 단위 · 외부 상태 대조
검증	테스트 · 검사 연결 지점	보호된 인수 기준 · PASS 조건
기록	이벤트 · Trace · 구성 이력	증거 연결 · 가림 · 보존 · 접근

제품을 선택한 것과 업무용 Harness 설계를 끝낸 것은 다르다.

통합 루프: 여섯 책임의 연결 순서

모델의 제안은 입력일 뿐, 실행과 완료 판정의 최종 권한이 아니다.

구조 설명용 의사코드

```
while budget.can_continue(state):
    ctx = build_context(instructions, state)
    action = model.propose(ctx)
    decision = gate(action, state)
    record("policy", action, decision)
    if decision != ALLOW:
        return pause_or_stop(state, decision)
    save_intent(state, action)
    result = execute_or_reconcile(action)
    state = update_and_checkpoint(state, result)
    verdict = verify(state)
    record("verification", action, verdict)
    if contract_complete(state, verdict):
        return finish_with_evidence(state)
    return escalate_with_evidence(state)
```

모델의 제안, 실행 허용, 결과 검증, 업무 완료를 분리한다.

01 입력 | 지침 + 선택된 상태

필요한 코드·오류·근거를 주입한다.

02 실행 | 제어 + 도구 + 상태

예산과 승인 검사 후 실행한다.
불명확한 결과는 별도 조정한다.

03 판정 | 검증 + 기록

미확인을 성공으로 만들지 않는다.
계약 범위까지 확인해 종료한다.

Claude Code Hook의 통제 지점

실행 전 판단과 실행 후 관측을 각 이벤트에 연결한다.

구조 설명용 예시

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason":
      "Outside the approved task scope"
  }
}
```

Hook의 반환 형식 예시

전체 보안 설정이나 스크립트가 아니다.

01

PreToolUse | 실행 전에 검사

도구와 인자를 검사해 차단할 수 있다.
검사 밖 경로와 실패 동작도 시험한다.

02

PostToolUse | 성공한 실행 뒤 관측

실행 결과를 기록·검사하는 지점이다.
이미 한 행동을 사전에 막지는 못한다.

03

제품별 적용 | 설치 버전과 조직 정책 확인

실제 자격증명·격리도 제한한다.
Hook만으로 전체 보안 경계는 아니다.

중요한 실행이 검사를 거치고, 실패하면 적절히 멈추는지 확인한다.

LangGraph: 저장·중단·재개

프레임워크 기능과 애플리케이션의 승인·외부 상태 책임을 나눈다.

구조 설명용 예시

```
from langgraph.types import interrupt, Command

def approval_node(state):
    answer = interrupt({
        "action": state["pending_action"]
    })
    return {"approval_answer": answer}

# checkpointer를 연결해 컴파일한 graph
config = {"configurable": {"thread_id": "RSV-R3"}}
graph.invoke(Command(resume=True), config=config)
```

01 저장 | 체크포인트와 스레드

같은 thread_id로 상태를 복원하고 작업을 재개한다.

02 재개 | 중단 노드의 재실행

중단 전 쓰기는 중복될 수 있다. 실행 분리와 멍등성을 검토한다.

03 권한 | 재개 입력과 인 증은 별개

서버에서 승인 주체·대상을 검증한다. 실행 직전에 정책을 재검사한다.

중단·재개 기능만으로 인증·승인·중복 방지·외부 복원이 완성되지는 않는다.

Multi-Agent에서도 공통 경계가 먼저다

역할을 나누는 것은 실행 권한을 늘리는 것이 아니다.

분배 | Orchestrator

하위 목표와 예산 배분 · 공유 정책 참조

작업 | Worker A · B

허용 도구와 작업공간 · 개별 상태와 증거

통합 | 결과 병합

revision 충돌 확인 · 미완료 작업 식별

판정 | 최종 검증

공통 완료 기준 · 권한·부작용·근거

01

경계 | 도구·자격증명은 최소 범위

다른 Worker를 통해 정책을 우회하지 않는다.

02

상태 | 소유자와 변경 버전을 명시

같은 파일과 외부 작업의 중복을 관리한다.

03

평가 | 조정 비용도 함께 측정

통신·충돌·재시도·통합 실패를 본다.

에이전트 수보다 지침·권한·상태·검증·기록의 경계가 먼저다.

Harness의 6개 책임 상태

각 책임의 담당자·산출물·검사 기준과 구성 버전을 함께 유지한다.

책임	운영에서 답할 질문	확인할 기준
지침	목표·범위·완료 조건은 무엇인가?	검토된 계약과 요구사항
도구	관측과 변경을 어떻게 실행하는가?	입출력·실패 의미가 명확한 도구
실행 제어	무엇을 허용·승인·중단하는가?	실제 실행 경로의 정책 검사
상태	재개할 때 무엇을 복원·재확인하나?	버전·의도·결과·외부 상태 대조
검증	어떤 근거로 완료를 인정하는가?	잘못된 증거로 통과하지 않음
기록	실패 원인과 변경을 설명할 수 있는가?	연결된 Trace·증거·구성 이력

Harness는 여섯 책임을 연결해 운영하고, 실패를 근거로 개선하는 구조다.